STUDENT EXPENSE TRACKER

VITyarthi – Build Your Own Project

Course: Programming / Problem Solving
Project Type: Python Console Application
Prepared by: _____
Programme: B.Tech CSE – AI & ML
Semester: I

1. Introduction

Student Expense Tracker is a lightweight Python application designed to help college students record and understand their daily spending. The system stores each expense with a date, category, amount, and description, then provides a complete list and summary by category and month.

The project follows the supplied VITyarthi requirement that a project should identify a meaningful problem, design a technical solution, implement it using course concepts, and demonstrate understanding through documentation and evaluation. ■filecite■turn0file0■L14-L20■

2. Problem Statement

College students make frequent small payments for food, transportation, education, shopping and entertainment. When these transactions are not recorded, it becomes difficult to know where money is being spent. The proposed system provides a simple, low-cost and easy-to-use way to record and analyse expenses.

3. Objectives

- Record student expenses in a structured format.
- Validate user input to reduce data-entry errors.
- Store records persistently in a CSV file.
- View and delete expense records.
- Calculate total, category-wise and monthly spending.
- Demonstrate modular programming and testing.

4. Scope

The current version is a console-based application. It does not include online banking integration, authentication, cloud storage, or automatic bank transactions. These can be future enhancements.

5. Functional Requirements

The supplied guidelines require at least three major functional modules and a clear input/output workflow. ■filecite■turn0file0■L24-L37■

Module	Function	Input	Output
Expense Management	Add/Delete/View expenses	Date, category, amount, description or ID	Updated expense list
Storage	Save and load CSV data	Expense records	Persistent records
Analytics	Calculate summaries	Expense list	Total/category/monthly totals
Validation	Check inputs	Date/category/amount	Valid value or error

6. Non-Functional Requirements

The supplied guidelines ask for at least four non-functional requirements such as performance, security, usability, reliability, maintainability and error handling.

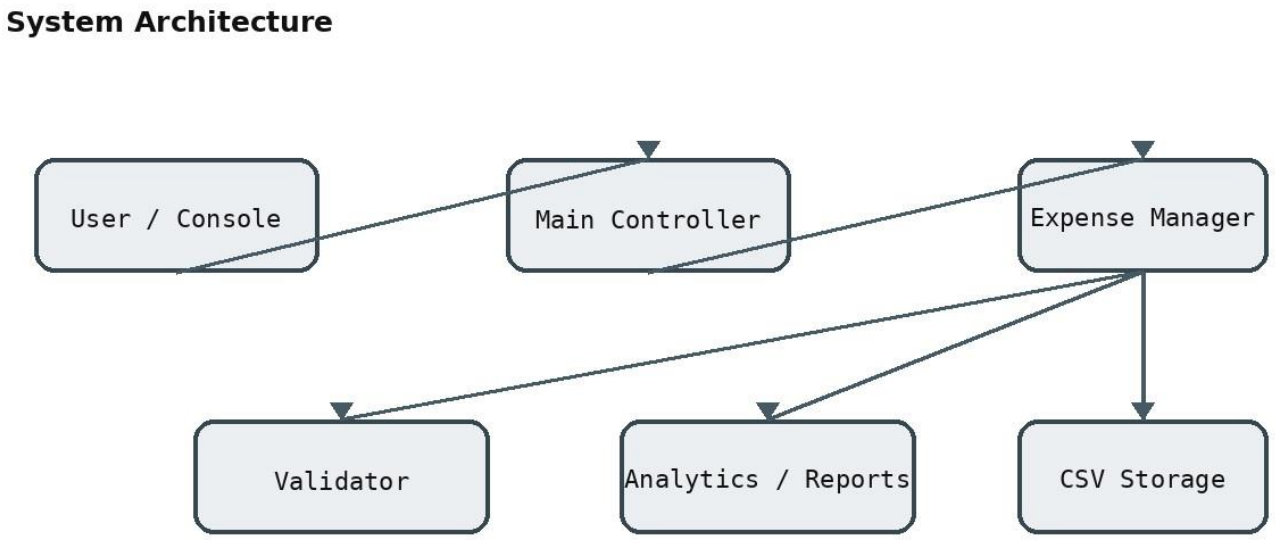
■filecite■turn0file0■L38-L50■

- Usability: menu-driven interface with simple prompts.
- Performance: in-memory calculations are fast for normal student-sized datasets.
- Reliability: CSV persistence keeps data between program runs.
- Maintainability: logic is divided into small modules.
- Error handling: invalid dates, categories, amounts and menu choices are handled without crashing themain loop.
- Resource efficiency: no external database or third-party library is required.

7. System Architecture

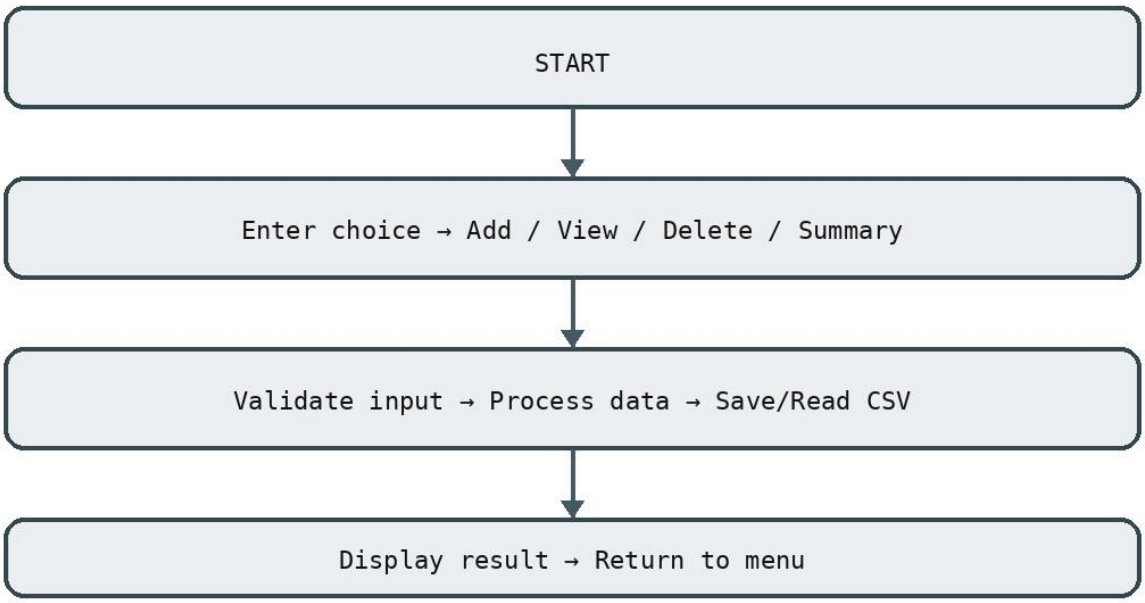
The architecture separates user interaction, business logic, validation, analytics and storage. This modular structure matches the technical expectation for clean implementation and appropriate use of programming concepts.

■filecite■turn0file0■L51-L63■



8. Process / Workflow

Workflow Diagram



9. Use Case Description

Actor	Use Case	Description
Student	Add Expense	Enters date, category, amount and description.
Student	View Expenses	Displays stored expense records and total.
Student	Delete Expense	Removes a record using its ID.
Student	View Summary	Displays category-wise and monthly spending.

10. Class / Component Design

Expense – data model containing ID, date, category, amount and description.
ExpenseStorage – reads/writes CSV data.
ExpenseManager – performs add, delete and retrieval operations.
Validator – validates date, amount and category.
Analytics – calculates totals and summaries.
Reports – formats output for the user.
Main – controls the application workflow.

11. Sequence of a Typical Add Operation

Student → Main Menu → ExpenseManager → Validator → Expense object → CSV Storage → Success message. If validation fails, the error is returned to the main menu and no invalid record is saved.

12. Storage / ER-Style Schema

Field	Type	Description
id	Integer	Unique expense identifier
date	YYYY-MM-DD	Date of expense
category	Text	Expense category
amount	Float	Expense amount
description	Text	Short description

The storage is a single CSV relation named **expenses**. The guidelines require database/storage design when storage is applicable. ■filecite■turn0file0■L78-L80■

13. Design Decisions & Rationale

Python was selected because it is suitable for introductory programming, file handling, functions, classes and testing. CSV was selected instead of a database to keep the first version simple and transparent. A modular folder structure was used so each responsibility can be understood and tested independently.

14. Implementation Details

The project contains more than the minimum 5–10 meaningful modules/files expected for a coding project: model, validation, storage, expense management, analytics, reporting, utilities, main program and tests.

■filecite■turn0file0■L60-L63■

File	Purpose
src/main.py	Main menu and program control
src/models.py	Expense data class
src/validator.py	Input validation
src/storage.py	CSV read/write
src/expense_manager.py	CRUD/business logic

src/analytics.py	Totals and summaries
src/reports.py	Formatted reports
src/utils.py	Menu utility
tests/test_expense_tracker.py	Automated validation tests

15. Source Code – Core Program

The complete source code is included in the accompanying project ZIP. The main execution command is **python -m src.main**. The code is intentionally written without external packages so it is easy to run in a college lab or Google Colab/local Python environment.

16. Sample Input / Output Screenshots

Sample Input / Main Menu

```
========== STUDENT EXPENSE TRACKER ==========
1. Add Expense
2. View Expenses
3. Delete Expense
4. View Summary
5. Exit
============================================
Enter choice: 1
Date (YYYY-MM-DD): 2026-09-29
Categories: Food, Travel, Education, Shopping, Entertainment, Other
Category: Food
Amount: 90
Description: Snacks
✓ Expense added successfully.
```

Sample Output / Expense List

```
ID | Date       | Category       | Amount | Description
-----------------------------------------------------------------
1  | 2026-09-25 | Food           | ₹120.00 | Lunch
2  | 2026-09-26 | Travel         | ₹ 80.00 | Bus fare
3  | 2026-09-27 | Education      | ₹250.00 | Notebook and printouts
4  | 2026-09-28 | Entertainment  | ₹150.00 | Movie
5  | 2026-09-29 | Food           | ₹ 90.00 | Snacks

Total: ₹690.00
```

16. Sample Input / Output Screenshots (continued)

Sample Output / Analytics

```
TOTAL SPENDING: ₹690.00

CATEGORY SUMMARY
Education        ₹250.00
Entertainment    ₹150.00
Food             ₹210.00
Travel           ₹80.00

MONTHLY SUMMARY
2026-09     ₹690.00
```

Sample Input / Error Handling

```
Enter choice: 1
Date (YYYY-MM-DD): 29-09-2026
Categories: Food, Travel, Education, Shopping, Entertainment, Other
Category: Food
Amount: 100
Description: Breakfast
Error: Date must be in YYYY-MM-DD format.
```

17. Testing Approach

Two automated tests are included: (1) adding multiple expenses and checking the total, and (2) checking category-wise aggregation. Manual validation cases cover invalid date format, invalid category, non-positive amount, invalid menu option and unknown deletion ID.

Test Case	Input	Expected Result	Status
TC01	Add valid Food expense	Record saved	Pass
TC02	Invalid date	Error shown; no save	Pass
TC03	Amount = 0	Validation error	Pass
TC04	Delete existing ID	Record removed	Pass
TC05	View summary	Totals displayed	Pass

18. Challenges Faced

- Designing a simple data format that remains readable.
- Keeping validation separate from business logic.
- Maintaining records after restarting the application.
- Formatting reports so that students can understand spending quickly.

19. Learnings & Key Takeaways

This project demonstrates practical use of Python classes, functions, modules, file handling, exception handling, data structures, validation, aggregation and unit testing. It also shows how a real-world problem can be converted into functional requirements and a small software architecture.

22. Conclusion

Student Expense Tracker is a compact but complete academic project that connects programming concepts with a practical student problem. It provides data entry, persistence, CRUD-style operations, validation and analytics while remaining easy to understand and demonstrate.

23. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (supplieddocument).
2. Python 3 standard library concepts: csv, dataclasses, pathlib, collections and unittest.
3. Project source files included with this report.